



UNIVERSIDAD
TECNOLÓGICA
METROPOLITANA
del Estado de Chile

INFB6074 - INFRAESTRUCTURA PARA C. DE DATOS
INGENIERÍA CIVIL EN CIENCIA DE DATOS

ACTIVIDAD SEMANA 4:

*“Tratamiento de Archivos Big Data,
Particionamiento y Selección Arquitectónica en
Python”*

Dr. Ing. Michael Miranda Sandoval

29 de abril de 2026

ACTIVIDAD SEMANA 4:

Integrantes (individual o parejas, máximo 2 integrantes):

Nombre: _____ **Email:** _____

Nombre: _____ **Email:** _____

- **Inicio formal de la actividad:** Semana 4 - Clases 1 y 2.
- **Modalidad:** Individual o parejas (máximo 2 integrantes).
- **Duración estimada:** 1 semana.

1. Instrucciones

Esta actividad está diseñada para que el estudiante analice, implemente y evalúe estrategias de tratamiento de archivos de gran tamaño en escenarios Big Data, articulando la clasificación de arquitecturas con decisiones técnicas sobre lectura incremental, particionamiento, formatos de almacenamiento y selección de infraestructura.

El foco de la semana no es solo procesar un archivo grande, sino comprender por qué las arquitecturas Big Data se clasifican según volumen, velocidad, variedad, tipo de procesamiento, modelo de almacenamiento, criticidad temporal y patrón de integración. A partir de una experiencia práctica en Python, el estudiante debe justificar cuándo basta una solución batch local, cuándo conviene particionar datos, cuándo un formato columnar aporta valor, cuándo se justifica un data lake o lakehouse, y cuándo tendría sentido escalar hacia una arquitectura distribuida, cloud o híbrida.

Indicaciones generales:

- La actividad puede ser realizada de forma individual o en parejas, con un máximo de 2 integrantes.
- Siga las tres partes de la actividad: investigación teórica, aplicación práctica en Python y síntesis del informe final.
- El trabajo debe abordar los dos ejes de la semana: clasificación de arquitecturas Big Data y selección de infraestructura mediante criterios técnicos.
- Utilice Python para construir, leer, procesar y comparar archivos de gran tamaño bajo restricciones de memoria.
- Use al menos las librerías `pandas`, `numpy`, `matplotlib` o `seaborn`, y una librería para formato columnar, como `pyarrow` o `fastparquet`.
- Puede incorporar `psutil`, `dask`, `polars` u otras librerías si aportan valor al análisis, pero no son obligatorias.
- Los datos pueden ser simulados por el grupo, pero deben representar un escenario organizacional plausible y estar justificados.
- El código debe permitir reproducir los resultados: generación o lectura de datos, procesamiento por fragmentos, agregaciones incrementales, conversión de formatos, particionamiento, mediciones y visualizaciones.
- Cite correctamente todas las fuentes consultadas y declare el uso de herramientas de apoyo como IA, notebooks de referencia o repositorios externos.
- Entregue un informe claro y bien organizado, código funcional, resultados reproducibles y visualizaciones integradas.

Lee atentamente cada sección y organiza tu trabajo para cubrir los objetivos de aprendizaje de la semana: clasificación de arquitecturas Big Data; dimensiones de volumen, velocidad y variedad; batch, streaming y near real time; arquitecturas centralizadas, distribuidas y federadas; data lake, data warehouse y lakehouse; criterios funcionales y no funcionales; CAPEX versus OPEX; on-premise, cloud e

híbrido; particionamiento; formatos de almacenamiento; complejidad operativa; y justificación técnica basada en evidencia.

2. Etapas de la Actividad

1. **Parte 1 - Investigación teórica:** Elaborar un documento técnico que explique cómo se clasifican las arquitecturas Big Data y cómo el tamaño, formato, particionamiento y patrón de acceso a los datos influyen en la selección de infraestructura.
2. **Parte 2 - Aplicación práctica en Python:** Implementar experimentos de lectura completa, lectura por partes, procesamiento incremental, conversión a formato columnar y particionamiento de archivos.
3. **Parte 3 - Informe y análisis final:** Sintetizar los hallazgos en un informe técnico con metodología, resultados, interpretación, discusión de trade-offs y recomendación arquitectónica para distintos escenarios.

Detalles de la parte práctica:

- 2.1. **Configuración del entorno experimental:** Documentar el sistema usado para ejecutar el análisis: versión de Python, sistema operativo, memoria RAM disponible, espacio en disco, librerías utilizadas, estructura del repositorio y fecha de ejecución. Se debe indicar explícitamente si el equipo permite o no cargar el archivo completo en memoria.
- 2.2. **Experimento A - Generación o preparación de un archivo de gran tamaño:** Construir o seleccionar un conjunto de datos tabular de tamaño suficiente para representar un escenario Big Data en escala local. El archivo puede ser sintético, pero debe simular un caso plausible, por ejemplo logs de transacciones, eventos de usuarios, sensores IoT, registros académicos, operaciones financieras o mediciones industriales. El dataset debe incluir al menos variables temporales, categóricas y numéricas. Se debe almacenar inicialmente en formato CSV o JSONL, y registrar tamaño en disco, número de filas, número de columnas y tipos de datos.
- 2.3. **Experimento B - Lectura monolítica versus lectura por partes:** Comparar una lectura tradicional del archivo completo con `pandas.read_csv()` contra una lectura incremental usando `chunks`. El objetivo es evaluar tiempo de lectura, uso aproximado de memoria, estabilidad de ejecución y límites prácticos del enfoque monolítico. Si el equipo no permite cargar el archivo completo, debe explicarse la falla o limitación observada y continuar el análisis mediante lectura por partes.
- 2.4. **Experimento C - Procesamiento incremental de datos grandes:** Implementar un pipeline que procese el archivo por fragmentos sin cargarlo completo en memoria. El pipeline debe incluir al menos: filtrado de registros, selección de columnas, conversión de tipos, agregación por grupo y cálculo de una métrica derivada. El resultado final debe combinar correctamente los resultados parciales de cada fragmento. Se espera que el estudiante demuestre comprensión del patrón *split-process-combine* aplicado a archivos grandes.
- 2.5. **Experimento D - Particionamiento, formato de almacenamiento y decisión arquitectónica:** Convertir el conjunto de datos original a un formato más adecuado para analítica, por ejemplo Parquet, y particionarlo por una variable relevante, como fecha, categoría, región, fuente o tipo de evento. Luego comparar la lectura y consulta entre el archivo original y la versión particionada. A partir de los resultados, justificar qué arquitectura sería más pertinente para distintos escenarios: procesamiento batch centralizado, data lake, lakehouse, arquitectura distribuida o solución cloud/híbrida.

Requisitos adicionales:

- Trabajar con un archivo suficientemente grande para evidenciar diferencias entre lectura completa y lectura por partes. El tamaño puede ajustarse a las capacidades del equipo, pero debe justificarse.
- El dataset debe tener al menos 1 millón de filas o, si el equipo no lo permite, una justificación técnica del tamaño usado.
- Comparar al menos dos estrategias de lectura: carga completa y carga por fragmentos con `chunksize`.
- Implementar al menos una agregación incremental donde los resultados parciales se combinen correctamente.
- Medir tiempo de ejecución y uso aproximado de memoria en cada estrategia.
- Evaluar al menos tres tamaños de `chunksize` distintos.
- Convertir el dataset original a un formato columnar, preferentemente **Parquet**.
- Implementar al menos una estrategia de particionamiento por una variable relevante.
- Comparar consulta o agregación sobre archivo original versus archivo particionado.
- Incluir al menos 4 visualizaciones: tiempo de lectura, uso de memoria, comparación de `chunksize` y diferencia entre formatos o particiones.
- Relacionar los resultados con una decisión arquitectónica: cuándo basta un procesamiento batch local, cuándo conviene particionar, cuándo se justifica un data lake/lakehouse y cuándo tendría sentido distribuir el procesamiento.

* Ejemplos referenciales por experimento

Los siguientes ejemplos son solo orientativos. Su función es ayudar a delimitar el tipo de problema esperado, pero cada grupo debe definir su propio dataset, tamaño, variables, parámetros y forma de análisis.

- **Ejemplo para el Experimento A:** Generar un archivo CSV con registros de eventos de usuarios, incluyendo columnas como `user_id`, `timestamp`, `event_type`, `region`, `device`, `amount` y `session_duration`. El archivo debe ser lo bastante grande para que su carga completa consuma memoria de forma apreciable.
- **Ejemplo para el Experimento B:** Comparar `pd.read_csv("eventos.csv")` contra `pd.read_csv("eventos.csv", chunksize=100000)`. Registrar tiempo de lectura, memoria estimada y comportamiento del equipo. Analizar por qué cargar todo el archivo puede ser inviable en escenarios Big Data.
- **Ejemplo para el Experimento C:** Procesar el archivo por partes para calcular ventas totales por región, número de eventos por tipo, promedio de duración de sesión por dispositivo o detección de registros anómalos. Cada fragmento debe producir resultados parciales, que luego deben combinarse en una tabla final coherente.
- **Ejemplo para el Experimento D:** Convertir el archivo CSV a **Parquet** y particionarlo por mes o región. Luego comparar una consulta que lea todo el archivo contra una consulta que solo acceda a una partición relevante. El análisis debe explicar cómo el formato columnar y el particionamiento influyen en una arquitectura tipo data lake o lakehouse.

* Registro mínimo obligatorio

Toda entrega debe registrar explícitamente:

- versión de Python utilizada;
- sistema operativo;

- memoria RAM aproximada y espacio disponible en disco;
- librerías empleadas;
- tamaño del archivo usado;
- número de filas y columnas;
- formato original del archivo;
- formatos alternativos generados, por ejemplo **Parquet**;
- criterio de particionamiento utilizado;
- tamaños de **chunksize** evaluados;
- tiempos obtenidos;
- uso aproximado de memoria;
- resultados de agregaciones incrementales;
- interpretación de resultados;
- recomendación arquitectónica final.

*** Advertencia de integridad académica y uso de herramientas de IA**

Los ejemplos de código, criterios y escenarios incorporados en esta guía son **solo referenciales**. Su propósito es orientar la forma general del problema y ayudar a encuadrar el laboratorio. No deben entenderse como una solución completa ni como una plantilla para copiar.

Cada grupo debe construir su propio desarrollo, lo que implica:

- proponer su propio escenario de datos;
- definir sus propias variables;
- justificar el tamaño del archivo utilizado;
- seleccionar sus propios tamaños de **chunksize**;
- definir su propia estrategia de particionamiento;
- interpretar con autonomía los resultados obtenidos;
- redactar conclusiones coherentes con la evidencia generada.

El uso de herramientas de inteligencia artificial no está permitido como sustituto del razonamiento humano del grupo. En particular, se penalizará subir tal cual las instrucciones del laboratorio a cualquier sistema de IA y presentar luego resultados, código, análisis o conclusiones entregadas de manera determinista, genérica o sustancialmente similares a las de otros grupos.

Se espera que, si un grupo utiliza alguna herramienta de apoyo, exista trabajo intelectual propio, adaptación crítica, reformulación de supuestos, selección justificada de ejemplos y análisis genuino. La simple delegación de la tarea a una IA no constituye aprendizaje ni trabajo válido para esta asignatura. No se aceptarán entregas esencialmente idénticas entre grupos, aunque existan cambios superficiales de nombres de variables, comentarios, orden del código o formato del texto. Si se detecta **alta similitud** entre ejercicios de distintos grupos, ya sea por copia directa o por uso acrítico de IA que produzca respuestas comunes, **todo el conjunto de estudiantes involucrados será sancionado con nota 1.0**.

3. Productos Esperados

Los entregables deben evidenciar la ejecución completa de la investigación teórica, la implementación práctica en Python y el análisis de resultados. Se espera un conjunto ordenado que permita reproducir los experimentos, validar las conclusiones y justificar decisiones de infraestructura Big Data con base técnica.

- Informe técnico en PDF con análisis conceptual, metodología, resultados, visualizaciones y conclusiones.
- Código fuente en Python, en formato `.py` o `.ipynb`, que implemente los experimentos de lectura, procesamiento incremental, conversión de formato y particionamiento.
- Datos de entrada, ya sean simulados o construidos por el grupo, en formato CSV, JSONL, Parquet o equivalente.
- Tablas de tiempos, uso aproximado de memoria, tamaños de archivo, resultados de agregación y comparaciones entre estrategias.
- Visualizaciones de tiempo de lectura, memoria, tamaños de `chunksize`, formatos de almacenamiento y particiones.
- Documentación del entorno experimental, incluyendo librerías, fecha de ejecución y estructura del repositorio.
- Repositorio con todos los archivos de entrega y un README con instrucciones claras de ejecución.

3.1 Estructura mínima del informe final

- Portada:** Título, nombre(s), fecha, curso y actividad.
- Resumen Ejecutivo:** Síntesis de los objetivos, metodología, principales hallazgos y recomendación general.
- Marco Teórico:** Arquitecturas Big Data, criterios de clasificación, volumen, velocidad, variedad, batch, streaming, near real time, data lake, data warehouse, lakehouse, cloud, on-premise e híbrido.
- Fundamento Técnico del Problema:** Tratamiento de archivos grandes, memoria limitada, lectura incremental, formatos row-oriented y column-oriented, particionamiento y patrones de acceso.
- Metodología Experimental:** Definición del dataset, variables, tamaño, formato original, estrategia de generación o selección, tamaños de `chunksize`, métricas y criterios de comparación.
- Implementación en Python:** Descripción del código, estructura de datos, lectura completa, lectura por partes, agregación incremental, conversión a Parquet y particionamiento.
- Resultados:** Tablas, métricas y gráficos correspondientes a los experimentos A, B, C y D.
- Análisis y Discusión:** Interpretación de resultados, trade-offs entre lectura completa y lectura incremental, efecto del formato, efecto del particionamiento, limitaciones del experimento y relación con Big Data.
- Decisiones Técnicas por Escenario:** Recomendación razonada para procesamiento batch local, data lake, lakehouse, arquitectura distribuida y solución cloud/híbrida.
- Conclusiones y Recomendaciones:** Hallazgos finales, limitaciones del trabajo y sugerencias de mejora.
- Anexos y Referencias:** Código relevante, datos crudos, comandos de ejecución y fuentes consultadas.

3.2 Estructura mínima del repositorio y entrega

- a. **README:** Instrucciones para instalar dependencias, generar o descargar datos, ejecutar el código y reproducir los resultados.
- b. **Código Python:** Scripts o notebooks organizados y documentados.
- c. **Datos:** Archivos originales, archivos convertidos, particiones y muestras si el tamaño total impide subir el dataset completo.
- d. **Resultados:** Tablas generadas por el código, tiempos, uso aproximado de memoria y resultados de agregaciones.
- e. **Visualizaciones:** Gráficos comparativos de lectura, memoria, formatos, particionamiento y rendimiento.
- f. **Entorno experimental:** Registro de versión de Python, sistema operativo, RAM aproximada, espacio en disco y librerías utilizadas.
- g. **Informe final:** Documento PDF con la narrativa completa del trabajo.

3.3 Recomendaciones y observaciones adicionales

- a) Mantener una redacción formal, clara y precisa.
- b) Numerar todas las figuras, tablas y anexos, y referenciarlos en el texto.
- c) Revisar ortografía, coherencia y consistencia metodológica antes de la entrega.
- d) Justificar el tamaño del dataset usado y explicar si representa una escala Big Data local, reducida o simulada.
- e) Diferenciar entre limitaciones del equipo local y necesidades reales de una organización.
- f) Evitar conclusiones absolutas basadas en un solo equipo o una sola ejecución.
- g) Repetir las mediciones cuando sea posible y reportar variabilidad si se observa.
- h) Explicar claramente la diferencia entre formato de archivo, sistema de almacenamiento y arquitectura de datos.
- i) No confundir una herramienta puntual, como **pandas** o **pyarrow**, con una arquitectura completa.
- j) Citar las fuentes consultadas y declarar el uso de IA o código ajeno cuando corresponda.
- k) La copia de trabajo ajeno será considerada plagio y sancionada según normativa institucional.

4. Rúbrica de Evaluación

La evaluación considera la profundidad conceptual, la calidad de la implementación en Python, el rigor experimental, la calidad del análisis arquitectónico y la presentación profesional del trabajo.

Escala de evaluación:

- 90–100: Excelente.
- 80–89: Muy bueno.
- 70–79: Bueno.
- 60–69: Suficiente.
- Menos de 60: Insuficiente.

Criterio	Excelente (4)	Bueno (3)	Suficiente (2)	Insuficiente (1)
A. Investigación teórica (25 %)				
Clasificación de arquitecturas Big Data	Precisa, completa y con comparación crítica.	Correcta y bien explicada.	Parcial con omisiones relevantes.	Incompleta o incorrecta.
Relación entre archivos grandes e infraestructura	Explica con rigor memoria, formatos, particionamiento y arquitectura.	Relaciona conceptos con ejemplos adecuados.	Presenta una relación superficial.	No demuestra comprensión aplicada.
B. Implementación en Python (25 %)				
Código funcional y organizado	Implementación robusta, clara, modular y reproducible.	Funcional con buena organización.	Funcional parcial o con errores menores.	No funcional o desordenada.
Lectura incremental y procesamiento por partes	Implementa correctamente lectura por fragmentos y combinación de resultados.	Implementa el proceso con detalles menores por mejorar.	Implementación básica o parcialmente correcta.	Lectura por partes incorrecta o ausente.
C. Diseño experimental (15 %)				
Dataset, tamaños y escenarios	Dataset plausible, tamaño justificado y escenario bien construido.	Buen diseño con detalles menores.	Diseño parcial o poco fundamentado.	Diseño insuficiente.
Comparación de formatos y particiones	Compara estrategias con criterios claros y evidencia suficiente.	Comparación adecuada.	Comparación limitada.	Comparación ausente o incorrecta.
D. Análisis de resultados (25 %)				
Interpretación de tiempos y memoria	Análisis profundo, cuantitativo y bien fundamentado.	Buen análisis con evidencia suficiente.	Análisis limitado o superficial.	Análisis pobre o ausente.
Discusión de trade-offs arquitectónicos	Relaciona resultados con batch, lake, lakehouse, cloud o distribución.	Relación válida con justificación aceptable.	Relación general poco precisa.	Sin análisis arquitectónico útil.
Recomendación técnica por escenarios	Conclusiones claras, aplicables y bien justificadas.	Conclusiones válidas con justificación aceptable.	Recomendaciones generales poco precisas.	Sin recomendaciones útiles.
E. Presentación y formato (10 %)				
Claridad y estructura del informe	Informe ordenado, profesional y coherente.	Informe adecuado con detalles menores.	Informe poco ordenado.	Informe desordenado o difícil de leer.
Visualizaciones y documentación	Gráficos claros, bien integrados y documentación completa.	Gráficos adecuados y documentación suficiente.	Gráficos limitados o documentación incompleta.	Sin visualizaciones relevantes o sin documentación.

Cuadro 1: Rúbrica de evaluación detallada - Actividad de tratamiento de archivos Big Data, particionamiento y selección arquitectónica

Referencias Bibliográficas

1. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
2. Hwang, K., Dongarra, J., & Fox, G. (2013). *Distributed and Cloud Computing: From Parallel Processing to the Internet of Things*. Morgan Kaufmann.
3. Tanenbaum, A. S., & Steen, M. van. (2017). *Distributed Systems* (3rd ed.). Pearson.
4. DAMA International. (2017). *DAMA-DMBOK: Data Management Body of Knowledge* (2nd ed.). Technics Publications.
5. Inmon, W. H. (2005). *Building the Data Warehouse*. Wiley.
6. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit* (3rd ed.). Wiley.
7. Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. *CIDR*.
8. White, T. (2015). *Hadoop: The Definitive Guide* (4th ed.). O'Reilly Media.
9. Python Software Foundation. *Python documentation*. Recuperado de <https://docs.python.org/3/>
10. Pandas Developers. *Pandas documentation*. Recuperado de <https://pandas.pydata.org/docs/>
11. NumPy Developers. *NumPy documentation*. Recuperado de <https://numpy.org/doc/>
12. Apache Arrow Developers. *Apache Arrow and Parquet documentation*. Recuperado de <https://arrow.apache.org/docs/>
13. Matplotlib Developers. *Matplotlib documentation*. Recuperado de <https://matplotlib.org/stable/>
14. Dask. *Dask documentation*. Recuperado de <https://docs.dask.org>
15. Fuentes oficiales adicionales, notas de clase y documentación técnica citadas explícitamente en el informe final.